Universität Rostock



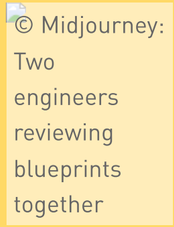
CODE REVIEW

Joern Ploennigs · AI4SC

Midjourney: Engineers reviewing technical blueprints

Hörsaalfrage

Würdet ihr eurem Kollegen vertrauen, wenn er euch Code gibt — ohne ihn zu lesen?



© Midjourney: Two engineers reviewing blueprints together

WARUM KI-CODE REVIEWEN?

KI ist zuversichtlich — nicht immer korrekt

KI ist gut in

- Code-Struktur und Syntax
- Standardmuster implementieren
- Erste Lösungen schnell liefern

KI macht Fehler bei

- Anforderungen vollständig umsetzen
- Randfälle berücksichtigen
- Nicht existierende Funktionen
- Typ-Annahmen validieren

Code Review: Drei Dimensionen

1. Anforderungsprüfung

Macht der Code, was verlangt wurde?

2. Designprüfung

Entspricht er der Architektur?

3. Implementierungsprüfung

Ist er korrekt, sicher, wartbar?

ANFORDERUNGSPRÜFUNG

Anforderungen systematisch prüfen

Für jede funktionale Anforderung drei Fragen:

Implementiert?

Gibt es Code, der diese Anforderung umsetzt?

Getestet?

Existiert ein Unit-Test dafür?

Bestanden?

Laufen alle Tests grün durch?

Anforderungs-Tracing: Beispiel

```
# Anforderung: Berechne Neigung in Prozent (Höhe / Länge × 100)

def berechne_neigung(hoehe, laenge):
    return hoehe / laenge   # Fehler: × 100 fehlt – Einheit falsch
```

Was prüfen?

Review-Fragen:

- [ ] Formel korrekt implementiert?
- [ ] Rückgabewert vollständig?
- [ ] Randfälle berücksichtigt?

Test schreiben:

```
# Erwartet: 10.0 (Prozent)
assert berechne_neigung(1, 10) == 10.0
# Ergebnis: 0.1 – Test schlägt fehl
```

DESIGNPRÜFUNG

Design-Match prüfen

↹

Modulstruktur

- Passt der Code ins richtige Modul?
- Werden bestehende Module genutzt?
- Entstehen zirkuläre Importe?

Klasseninterfaces

- Stimmen Klassennamen mit dem UML überein?
- Stimmen Methodensignaturen?
- Werden vorhandene Klassen erweitert?

UML-Design:

```
class Messung:
    + wert: float
    + einheit: str
    + validiere() → bool
```

KI-generierter Code:

```python
class DataPoint:       # falscher Name
    def __init__(self, v):
        self.val = v   # falsches Attribut
    # validiere() fehlt
```

IMPLEMENTIERUNGSPRÜFUNG

Typische Implementierungsprobleme

Division ohne Null-Prüfung

```python
def neigung(h, l):
    return h / l  # Fehler: Division durch Null!

# Fix:
def neigung(h, l):
    if l == 0:
        raise ValueError("Länge = 0")
    return h / l
```

Mutable Default-Argument

```python
def sammle(w, liste=[]):  # Fehler!
    liste.append(w)
    return liste

# Fix:
def sammle(w, liste=None):
    if liste is None:
        liste = []
    liste.append(w)
    return liste
```

Fehlende Typ-Annotationen

```python
def verarbeite(daten):  # Typ unklar
    return daten * 2

# Fix:
def verarbeite(daten: list[float]) -> list[float]:
    return [x * 2 for x in daten]
```

Hartcodierte Werte

```python
if temp > 100:  # Warum 100?
    alarm()

# Fix:
MAX_TEMP_CELSIUS = 100
if temp > MAX_TEMP_CELSIUS:
    alarm()
```

Implementierungs-Scan: Kernchecks

Was bei jedem Code-Block zu prüfen ist:

- [ ] Division: Nullprüfung vorhanden?
- [ ] Standardargumente: kein `list=[]` oder `dict={}`?
- [ ] Typ-Annotationen: alle Parameter annotiert?

Implementierungs-Scan: Ergänzungen

Danach die zweite Prüfrunde:

- [ ] `return`-Anweisung: in jeder Funktion vorhanden?
- [ ] Hartcodierte Werte: durch Konstanten ersetzt?
- [ ] Fehlerbehandlung: sinnvolle Exceptions definiert?

Hörsaalfrage

Findet in 2 Minuten mindestens drei Probleme:

```python
def mittelwert(werte=[]):
    summe = 0
    for wert in werte:
        summe += wert
    return summe / len(werte)
```

Nutzt die Checkliste:

- Anforderungen vollständig?
- Randfälle berücksichtigt?
- Implementierungsprobleme sichtbar?

Mögliche Treffer:

- Mutable Default-Argument
- Division durch Null bei leerer Liste
- Keine Typ-Annotationen

KI-HALLUZINATIONEN

KI-Halluzinationsmuster

Nicht existierende Methoden

```python
import pandas as pd

df = pd.read_csv("daten.csv")
df.interpolate_missing()  # existiert nicht
result = df.to_geojson()  # existiert nicht
```

Nicht existierende Module

```python
import geopandas_lite      # existiert nicht
import pandas.geospatial   # existiert nicht
```

Falsche API-Nutzung

```python
# sklearn falsch verwendet:
model.fit_transform(X_test)     # nur für Training
model.score(X, y, metric="f1")  # kein metric-Parameter
```

Unnötige Abstraktionen

```python
class SingletonLoader:   # für Einmalnutzung unnötig
    _instance = None
    def __new__(cls): ...
```

Erkennungsstrategie

Jeden `import` und jeden Methodenaufruf gegen die Dokumentation prüfen. Code ausführen — nicht nur lesen.

KI-EVALUIERUNGSCHECKLISTE

KI-Evaluierungscheckliste

Anforderungen & Design

- [ ] Anforderungsabdeckung: alle Anforderungen implementiert?
- [ ] Design-Übereinstimmung: Module, Klassen, Signaturen korrekt?

Implementierung

- [ ] Typ-Korrektheit: Annotationen vorhanden und korrekt?
- [ ] Randfälle: Division, leere Listen, None-Werte behandelt?
- [ ] Fehlerbehandlung: sinnvolle Exceptions definiert?

KI-spezifisch

- [ ] Keine halluzierten Aufrufe: alle Funktionen verifiziert?
- [ ] Test-Suite: alle Tests laufen durch?

Agentic Workflow nach der Generierung

> **Agentic workflow:** Nach jeder KI-Generierung: 1. Anforderungen → Checkliste durchgehen 2. Design → UML vergleichen 3. Implementierung → sechs Muster scannen 4. Halluzinationen → Importe und Aufrufe testen 5. Tests → alle grün? 6. Nein → Prompt präzisieren und neu generieren

Code Review = Verify

Agentic Cycle

Review schließt den Agentic Cycle:

```
Prompt
  ↓
Generate (KI)
  ↓
Understand
  ↓
Verify ← Code Review
  ↓
Iterate (bei Fehler)
```

Review schlägt fehl → iterieren:

- Spezifischen Fehler im Prompt benennen
- Checklisten-Punkt als Prompt-Ergänzung nutzen
- Neu generieren lassen
- Erneut reviewen

Merksatz

Tests prüfen automatisch. Code Review prüft, was Tests nicht sehen können.

QUESTIONS